

EC200A-CN(TA) QuecOpen

低功耗模式用户指导

LTE Standard 模块系列

版本：1.0.0

日期：2022-07-19

状态：临时文件



上海移远通信技术股份有限公司（以下简称“移远通信”）始终以为客户提供最及时、最全面的服务为宗旨。如需任何帮助，请随时联系我司上海总部，联系方式如下：

上海移远通信技术股份有限公司
上海市闵行区田林路 1016 号科技绿洲 3 期（B 区）5 号楼 邮编：200233
电话：+86 21 5108 6236 邮箱：info@quectel.com

或联系我司当地办事处，详情请登录：<http://www.quectel.com/cn/support/sales.htm>。

如需技术支持或反馈我司技术文档中的问题，请随时登陆网址：
<http://www.quectel.com/cn/support/technical.htm> 或发送邮件至：support@quectel.com。

前言

移远通信提供该文档内容以支持客户的产品设计。客户须按照文档中提供的规范、参数来设计产品。同时，您理解并同意，移远通信提供的参考设计仅作为示例。您同意在设计您目标产品时使用您独立的分析、评估和判断。在使用本文档所指导的任何硬软件或服务之前，请仔细阅读本声明。您在此承认并同意，尽管移远通信采取了商业范围内的合理努力来提供尽可能好的体验，但本文档和其所涉及服务是在“可用”基础上提供给您的。移远通信可在未事先通知的情况下，自行决定随时增加、修改或重述本文档。

使用和披露限制

许可协议

除非移远通信特别授权，否则我司所提供硬软件、材料和文档的接收方须对接收的内容保密，不得将其用于除本项目的实施与开展以外的任何其他目的。

版权声明

移远通信产品和本协议项下的第三方产品可能包含受移远通信或第三方材料、硬软件和文档版权保护的相关资料。除非事先得到书面同意，否则您不得获取、使用、向第三方披露我司所提供的文档和信息，或对此类受版权保护的资料进行复制、转载、抄袭、出版、展示、翻译、分发、合并、修改，或创造其衍生作品。移远通信或第三方对受版权保护的资料拥有专有权，不授予或转让任何专利、版权、商标或服务商标权的许可。为避免歧义，除了正常的非独家、免版税的产品使用许可，任何形式的购买都不可被视为授予许可。对于任何违反保密义务、未经授权使用或以其他非法形式恶意使用所述文档和信息的违法侵权行为，移远通信有权追究法律责任。

商标

除另行规定，本文档中的任何内容均不授予在广告、宣传或其他方面使用移远通信或第三方的任何商标、商号及名称，或其缩略语，或其仿冒品的权利。

第三方权利

您理解本文档可能涉及一个或多个属于第三方的硬软件和文档（“第三方材料”）。您对此类第三方材料的使用应受本文档的所有限制和义务约束。

移远通信针对第三方材料不做任何明示或暗示的保证或陈述，包括但不限于任何暗示或法定的适销性或特定用途的适用性、平静受益权、系统集成、信息准确性以及与许可技术或被许可人使用许可技术相关的不侵犯任何第三方知识产权的保证。本协议中的任何内容都不构成移远通信对任何移远通信产品或任何其他硬件、设备、工具、信息或产品的开发、增强、修改、分销、营销、销售、提供销售或以其他方式维持生产的陈述或保证。此外，移远通信免除因交易过程、使用或贸易而产生的任何和所有保证。

隐私声明

为实现移远通信产品功能，特定设备数据将会上传至移远通信或第三方服务器（包括运营商、芯片供应商或您指定的服务器）。移远通信严格遵守相关法律法规，仅为实现产品功能之目的或在适用法律允许的情况下保留、使用、披露或以其他方式处理相关数据。当您与第三方进行数据交互前，请自行了解其隐私保护和数据安全政策。

免责声明

- 1) 移远通信不承担任何因未能遵守有关操作或设计规范而造成损害的责任。
- 2) 移远通信不承担因本文档中的任何因不准确、遗漏、或使用本文档中的信息而产生的任何责任。
- 3) 移远通信尽力确保开发中功能的完整性、准确性、及时性，但不排除上述功能错误或遗漏的可能。除非另有协议规定，否则移远通信对开发中功能的使用不做任何暗示或法定的保证。在适用法律允许的最大范围内，移远通信不对任何因使用开发中功能而遭受的损害承担责任，无论此类损害是否可以预见。
- 4) 移远通信对第三方网站及第三方资源的信息、内容、广告、商业报价、产品、服务和材料的可访问性、安全性、准确性、可用性、合法性和完整性不承担任何法律责任。

版权所有 ©上海移远通信技术股份有限公司 2022，保留一切权利。

Copyright © Quectel Wireless Solutions Co., Ltd. 2022.

文档历史

修订记录

版本	日期	作者	变更表述
-	2022-01-24	Larry ZHANG	文档创建
1.0.0	2022-07-19	Larry ZHANG/ Eyelyn TANG	临时版本

目录

文档历史	3
目录	4
表格索引	5
图片索引	6
1 引言	7
2 低功耗模式	8
2.1. 低功耗模式介绍	8
2.2. GPIO 休眠唤醒方案	9
2.2.1. sleep_sys_ind	9
2.2.2. wakeup_out	9
2.2.3. wakeup_in	10
2.3. 低功耗模式 GPIO 配置	10
3 低功耗模式 API	12
3.1. 头文件	12
3.2. 函数概览	12
3.3. 函数详解	13
3.3.1. WakeLock 相关 API	13
3.3.1.1. ql_lpm_init	13
3.3.1.2. ql_autosleep_enable	14
3.3.1.3. ql_slp_wakelock_create	15
3.3.1.4. ql_slp_wakelock_lock	15
3.3.1.5. ql_slp_wakelock_unlock	16
3.3.1.6. ql_slp_wakelock_destroy	16
4 常见问题处理	17
4.1. 模块无法进入休眠	17
4.2. 唤醒原因排查	18
4.2.1. VBUS 中断唤醒	18
4.2.2. RTC 中断唤醒	18
4.2.3. Wakeup_in 中断唤醒	19
4.2.4. 电话/短信/网络消息等中断唤醒	19
5 附录 参考文档与术语缩写	20

表格索引

表 1: sleep_sys_ind 电平	9
表 2: wakeup_out 电平	9
表 3: wakeup_in 电平	10
表 4: GPIO 配置说明	10
表 5: 函数概览	12
表 6: 参考文档	20
表 7: 术语缩写	20

Preliminary Document
Not Checked

图片索引

图 1: 休眠唤醒流程.....	7
图 2: 低功耗模式状态图	8
图 3: GPIO 休眠唤醒方案.....	9
图 4: DTSI 说明	10

Preliminary Document
Not Checked

1 引言

移远通信 EC200A-CN(TA)模块支持 QuecOpen®方案。QuecOpen®是开源的基于 Linux 的嵌入式开发平台，可简化 IoT 应用的软件设计和开发过程。有关 QuecOpen 的详细信息，请参考文档 [1]。

本文档主要介绍基于 Linux 的 AutoSleep 和 WakeLock 技术开发的一套 QuecOpen®休眠唤醒框架。

- **AutoSleep:** Autosleep 从 Android wakelocks 集中演化而来，基于唤醒源实现，达到 Opportunistic suspend 的效果，意思是在“系统没有事情在做”时，将自动切换到低功耗状态。
- **WakeLock:** 唤醒锁机制。“系统没有事情在做”的判断依赖于 wakeup events framework, AutoSleep 开启后，只要系统没有正在处理和新增的唤醒源，就尝试将系统挂起，若挂起的过程中有事件产生，则会进入休眠唤醒流程。

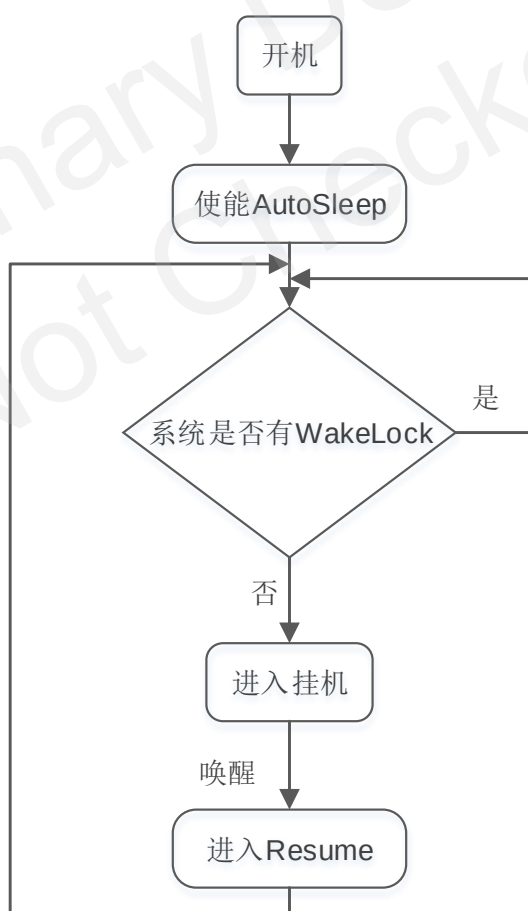


图 1：休眠唤醒流程

2 低功耗模式

2.1. 低功耗模式介绍

低功耗模式（即 LPM）的状态图如下所示。当 MCU/Android 检测车辆 CAN 信号发现熄火（即 ACC OFF）时，将通过 SPI、UART 和 USB 等接口通知模块停止用户应用程序的各项业务、停止外设工作、解锁相应的唤醒锁。当模块内部所有唤醒锁均被释放且所有外设都未工作时，模块将进入休眠，即低功耗模式。

当电话、短信、数据、定时器及特定 GPIO 中断等 5 种中断事件出现时，模块会从休眠状态中唤醒。相应的回调函数或者处理函数锁定相应的唤醒锁后，模块进入正常工作状态。

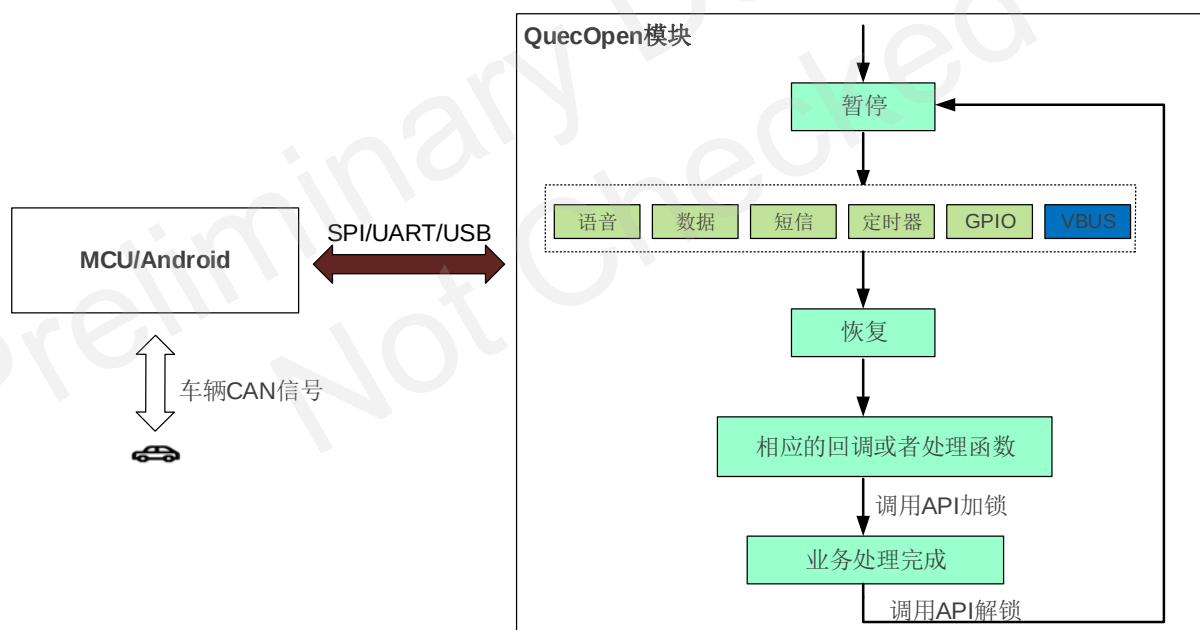


图 2：低功耗模式状态图

备注

建议模块控制的外设在休眠唤醒时正常供电。若外设必须要在休眠时去电、唤醒时上电，则要保证外设供电的控制模块侧，避免外部 MCU 等控制器上电和模块内驱动进入休眠的时间不同步，造成驱动休眠失败。

2.2. GPIO 休眠唤醒方案

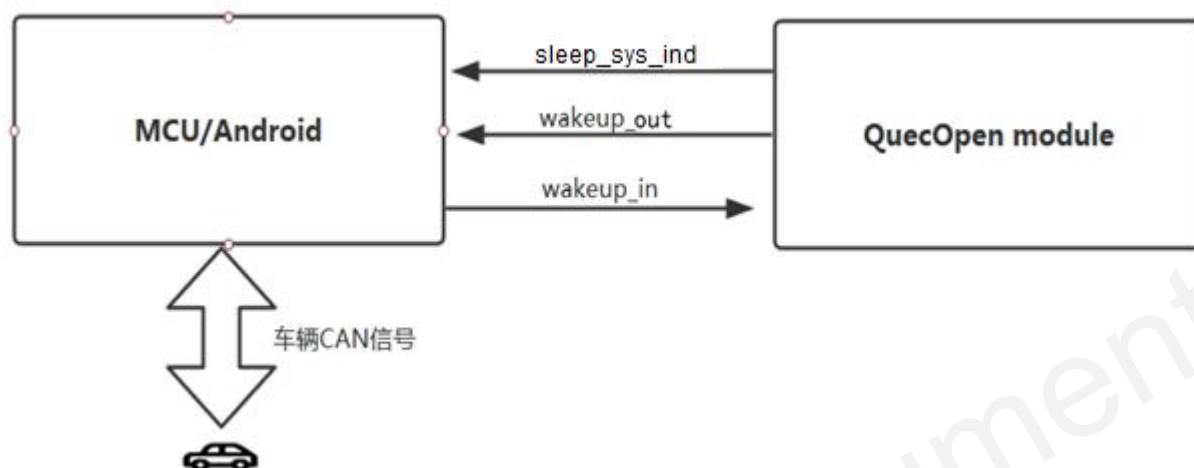


图 3: GPIO 休眠唤醒方案

2.2.1. sleep_sys_ind

sleep_sys_ind, 相当于 wakeup_sys_ind。该引脚由模块驱动注册管理, 表征系统进入或退出休眠。

表 1: sleep_sys_ind 电平

电平	表征状态
0	默认表示模块系统进入休眠
1	默认表示模块系统退出休眠

2.2.2. wakeup_out

该引脚由用户层注册管理, 表征用户应用程序进入或退出休眠 (具体使用的 GPIO 管脚由用户自行决定)。

表 2: wakeup_out 电平

电平	表征状态
0	默认表示应用程序进入休眠

1 默认表示应用程序退出休眠

2.2.3. wakeup_in

该引脚由模块驱动注册管理，表征外接 MCU 控制的模块休眠唤醒状态。

表 3: wakeup_in 电平

电平	表征状态
0	默认表示 MCU 控制模块休眠
1	默认表示 MCU 控制模块唤醒

2.3. 低功耗模式 GPIO 配置

DTS 配置在文件 `ql-ol-kernel/arch/arm/boot/dts/ql-asr1803-overlay.dtsi` 中。其具体配置示例如下：

```

quec,gpio_lpm{
    compatible = "quec,ql_lpm";
    pinctrl-names = "default","sleep";
    pinctrl-0 = <&wakeup_in_pin &sleep_sys_ind>;
    pinctrl-1 = <&wakeup_in_pin sleep>;
    gpios = <&gpio 117 0>,
           <&gpio 120 0>;
    ql,gpio-names = "wakeup_in","sleep_sys_ind";
    ql,sleep-sys-ind-enable;
    ql,sleep-sys-ind-state = <0>;
    status = "ok";
};
    
```

图 4: DTSI 说明

表 4: GPIO 配置说明

参数	含义	默认状态
gpios	wakeup_in 和 sleep_sys_ind 的 QuecOpen 引脚配置	wakeup_in: 117
		sleep_sys_ind: 120
ql,sleep-sys-ind-enable	是否开启休眠指示引脚	开启

ql,sleep-sys-ind-state	0	模块休眠时指示低电平，唤醒时指示高电平	0
	1	模块休眠时指示高电平，唤醒时指示低电平	
status	ok	表示开启低功耗模式	ok
	disabled	表示关闭低功耗模式	

Preliminary Document
Not Checked

3 低功耗模式 API

3.1. 头文件

低功耗模式 API 的头文件为 `ql_lpm.h` 和 `ql_sleep_wakelock.h`，位于 `ql-sysroots/usr/include/ql-sdk` 目录下，若无特别说明，本文档所提到的头文件均在该目录下。

低功耗模式 API 的使用可以参考 SDK 示例程序，SDK 示例程序为 `example_lpm.c`，位于 `sample/low_power_consume_app` 目录下。

3.2. 函数概览

表 5：函数概览

函数	说明
<code>ql_lpm_init()</code>	初始化低功耗模式
<code>ql_autosleep_enable()</code>	使能 AutoSleep
<code>ql_slp_wakelock_create()</code>	创建唤醒锁
<code>ql_slp_wakelock_lock()</code>	锁定已创建的唤醒锁
<code>ql_slp_wakelock_unlock()</code>	解锁指定的唤醒锁
<code>ql_slp_wakelock_destroy()</code>	注销唤醒锁

3.3. 函数详解

3.3.1. WakeLock 相关 API

3.3.1.1. ql_lpm_init

该函数用于初始化低功耗模式。

- 函数原型

```
int ql_lpm_init(ql_lpm_handler_t ql_lpm_handler)
```

- 参数

ql_lpm_handler:

[In] 回调函数，当 wakeup_in 引脚电平发生变化时将触发该回调函数。详见第 3.3.1.1.1 章。

- 返回值

0 函数执行成功

-1 函数执行失败

备注

用户重复调用该接口时，只有最后一次调用传入的回调函数会执行，因此不建议重复调用该接口。

3.3.1.1.1. ql_lpm_handler_t

当 wakeup_in 引脚的电平发生变化时将触发该回调函数，可通过 *ql_lpm_init()* 注册该回调函数。

- 函数原型

```
typedef void (*ql_lpm_handler_t)(ql_lpm_edge_t lpm_edge)
```

- 参数

lpm_edge:

[In] 底层上报的 wakeup_in 电平边沿变化。详见第 3.3.1.1.2 章。

- 返回值

无

3.3.1.1.2. ql_lpm_edge_t

wakeup_in 电平边沿变化枚举定义如下：

```
typedef enum
{
    E_QL_LPM_FALLING = 0,
    E_QL_LPM_RISING = 1,
}ql_lpm_edge_t
```

● 成员

成员	描述
<i>E_QL_LPM_FALLING</i>	下降沿（wakeup_in 电平下降唤醒模块或 wakeup_out 电平下降唤醒 MCU）
<i>E_QL_LPM_RISING</i>	上升沿（wakeup_in 电平上升唤醒模块或 wakeup_out 电平上升唤醒 MCU）

3.3.1.2. ql_autosleep_enable

该函数用于使能 AutoSleep，使能后系统会在满足条件后自动进入休眠。

● 函数原型

```
int ql_autosleep_enable(int enable)
```

● 参数

enable:

[In] 是否使能 AutoSleep。

0 关闭 AutoSleep

1 使能 AutoSleep

● 返回值

0 函数执行成功

-1 函数执行失败

备注

*ql_autosleep_enable(1)*使能 AutoSleep 后，一般不建议使用 *ql_autosleep_enable(0)*来保持唤醒，而应该调用 *ql_slp_wakelock_lock()*使系统保持唤醒，调用 *ql_slp_wakelock_unlock()*可以释放唤醒锁，放弃

休眠锁定。详情请参考第 3.3.1.4 章和第 3.3.1.5 章。系统休眠后进程会被冻结，唤醒后进程会继续运行。

3.3.1.3. ql_slp_wakelock_create

该函数用于创建唤醒锁。此接口创建的唤醒锁所有者为当前进程；最多可以创建 512 个休眠唤醒锁。

● 函数原型

```
int ql_slp_wakelock_create(const char *name, size_t len)
```

● 参数

name:

[In] 唤醒锁名称（在 `/sys/kernel/debug/wakeup_sources` 文件内体现）

len:

[In] 唤醒锁名称的长度，最大长度为 28 个字符。

● 返回值

唤醒锁描述符

函数执行成功

其他值

函数执行失败。错误码详见 `errno.h`。

3.3.1.4. ql_slp_wakelock_lock

该函数用于锁定已创建的唤醒锁，锁定后模块无法进入休眠模式。

● 函数原型

```
int ql_slp_wakelock_lock(int fd)
```

● 参数

fd:

[In] 唤醒锁描述符

● 返回值

0 函数执行成功

-1 函数执行失败

3.3.1.5. ql_slp_wakelock_unlock

该函数用于解锁指定的唤醒锁。若 Linux 系统内没有其他程序持有唤醒锁，且程序已使能 AutoSleep，则模块会自动进入休眠。

- 函数原型

```
int ql_slp_wakelock_unlock(int fd)
```

- 参数

fd:

[In] 唤醒锁描述符

- 返回值

0 函数执行成功

-1 函数执行失败

3.3.1.6. ql_slp_wakelock_destroy

该函数用于注销唤醒锁。

- 函数原型

```
int ql_slp_wakelock_destroy(int fd)
```

- 参数

fd:

[In] 唤醒锁描述符

- 返回值

0 函数执行成功

-1 函数执行失败

4 常见问题处理

4.1. 模块无法进入休眠

系统进入休眠时采用持锁机制，当使能 AutoSleep 后，若某个应用还持有唤醒锁，整个系统都无法进入休眠；系统内所有唤醒锁都释放后，系统方可进入休眠。

若用户在应用程序中已经释放了唤醒锁，但模块仍未进入休眠模式，可执行 `awk '$7!= 0 {print $1" "$7}' /sys/kernel/debug/wakeup_sources` 查看当前系统持有的唤醒锁：

若输出 mv-udc 和 mv-otg，则表示 USB_VBUS 引脚处于高电平状态，需拉低 USB_VBUS 引脚电平使模块休眠。

```
root@openwrt:/# awk '$7!= 0 {print $1" "$7}' /sys/kernel/debug/wakeup_sources
name active_since
mv-udc 5102
mv-otg 5100
root@openwrt:/#
```

若输出 ql_lpm，则表示 lpm 程序持有唤醒锁，需释放唤醒锁使模块休眠。

```
root@openwrt:/data# awk '$7 !=0 {print $1" "$7}' /sys/kernel/debug/wakeup_sources
name active_since
3063.ql_lpm 4719
root@openwrt:/data#
```

若输出如下信息，则表示系统未持有唤醒锁，模块可以休眠。

```
root@openwrt:/data# awk '$7 !=0 {print $1" "$7}' /sys/kernel/debug/wakeup_sources
name active_since
root@openwrt:/data#
```

备注

若输出信息为其他唤醒锁标识符，用户可自行排查对应的应用程序内是否存在唤醒锁。只有将应用程序内所有的唤醒锁都释放了，模块才能正常进入休眠。

4.2. 唤醒原因排查

可使用模块的调试串口捕捉日志信息，通过日志信息对模块唤醒原因进行分析，确认是否为非预期唤醒，并针对性地处理。

如需排查唤醒原因，首先执行如下命令，打开调试串口 log，即可查看唤醒中断信息。

```
echo 9 > /proc/sys/kernel/printk
echo 16371 > /dev/msocket_dump
```

然后，根据第 4.2.1~4.2.4 章排查唤醒原因。如需移远通信技术支持分析唤醒原因，请提供休眠唤醒时的 Kernel Log；若为网络消息唤醒，请提供 CP Log。

4.2.1. VBUS 中断唤醒

VBUS 作为唤醒源唤醒的日志如下图所示：

```
[ 156.724616] Msock: msocket_read: portq_recv returns fffffe00
[ 156.730960] Msock: msocket_read: portq_recv returns fffffe00
[ 156.737578] Msock: msocket_read: portq_recv returns fffffe00
[ 156.842071] bridge0: port 2(eth0) entered disabled state
[ 156.851435] mv-udc mv-udc: enable irq wakeup
[ 156.851618] =====quectel_wakeup:=====
[ 156.854546] =====vbus=====
[ 156.854607] vbuson_work
[ 156.854637] no wakelock for suspend state
[ 156.854668] =====vbus=====
[ 156.854698] vbuson_work
[ 156.854729] no wakelock for suspend state
```

4.2.2. RTC 中断唤醒

RTC 作为唤醒源唤醒的日志如下图所示。模块有两个 RTC 设备，分别为 RTC0 和 RTC1：

```
[ 792.227193] printk: Suspending console(s) (use no_console_suspend to debug)
[ 792.322170] bridge0: port 2(eth0) entered disabled state
[ 792.324610] change ddr freq to lowest value. (cur: 266000khz)
[ 792.331472] mv-udc mv-udc: enable irq wakeup
[ 792.331625] =====wake up events status =====
[ 792.331625] BEFORE SUSPEND AWUCRS:0x90080000
[ 792.331625] Power Mode Status :0xdf
[ 792.331625] =====
[ 792.331625] =====quectel_wakeup:=====
[ 792.331686] PM: pm_system_irq_wakeup: 22 triggered rtc Alrm
[ 792.334278] =====rtc0=====
[ 792.334370] =====rtc0=====
```

```
[ 848.546893] printk: Suspending console(s) (use no_console_suspend to debug)
[ 848.641778] bridge0: port 2(eth0) entered disabled state
[ 848.644737] change ddr freq to lowest value. (cur: 266000khz)
[ 848.651630] mv-udc mv-udc: enable irq wakeup
[ 848.652026] =====wake up events status =====
[ 848.652026] BEFORE SUSPEND AWUCRS:0x90080000
[ 848.652026] Power Mode Status :0xdf
[ 848.652026] =====
[ 848.652026] =====quectel_wakeup:=====
[ 848.655351] =====rtc1=====
[ 848.656540] =====rtc1=====
[ 848.660383] change ddr freq to saved value. (cur: 533000khz)
[ 848.962149] emac_phy_connect: phy@0
```

4.2.3. Wakeup_in 中断唤醒

Wakeup_in 是模块的唤醒引脚，默认为 GPIO117，唤醒日志如下图所示：

```
[ 971.494616] (elapsed 0.001 seconds)
[ 971.499588] done.
[ 971.505413] printk: Suspending console(s) (use no_console_suspend to debug)
[ 971.611461] bridge0: port 2(eth0) entered disabled state
[ 971.614084] change ddr freq to lowest value. (cur: 266000khz)
[ 971.620947] mv-udc mv-udc: enable irq wakeup
[ 971.621099] =====wake up events status =====
[ 971.621099] BEFORE SUSPEND AWUCRS:0x90080000
[ 971.621099] Power Mode Status :0xdf
[ 971.621099] =====
[ 971.621099] =====quectel_wakeup:=====
[ 971.624058] =====wakeup_in=====
```

4.2.4. 电话/短信/网络消息等中断唤醒

常见电话、短信、网络消息等中断唤醒的日志为在 quectel_wakeup 关键词之后打印 16 进制数据，如下图所示：

```
[ 982.226736] printk: Suspending console(s) (use no_console_suspend to debug)
[ 982.331228] bridge0: port 2(eth0) entered disabled state
[ 982.333821] change ddr freq to lowest value. (cur: 266000khz)
[ 982.340683] mv-udc mv-udc: enable irq wakeup
[ 982.340836] =====wake up events status =====
[ 982.340836] BEFORE SUSPEND AWUCRS:0x90080000
[ 982.340836] Power Mode Status :0xc3
[ 982.340836] =====
[ 982.340836] =====quectel_wakeup:=====
[ 982.341049] portg RX: DUMP BEGIN port :1, Length:308 --
[ 982.341080] 00 00 00 00 01 00 00 00 00 00 00 00 24 01 00 00
[ 982.341110] 01 00 00 00 05 00 00 00 18 01 00 00 2d 00 00 00
[ 982.341141] 05 00 0b 00 10 00 00 00 d0 4e 35 07 8d 00 d0 81
[ 982.341171] 8a 81 03 01 25 00 82 02 81 82 85 13 80 00 28 00
[ 982.341171] 55 00 29 00 53 00 49 00 4d 53 61 5e 94 75 28 8f
```

具体的网络端唤醒场景需要通过 CP Log 排查，可联系移远通信技术支持协助排查具体原因。

5 附录 参考文档与术语缩写

表 6: 参考文档

文档名称

[1] Quectel_EC200A-CN(TA)_QuecOpen_快速开发指导

表 7: 术语缩写

缩写	英文全称	中文全称
API	Application Program Interface	应用程序接口
CAN	Controller Area Network	控制器局域网
CP	Control Program	控制程序
DTS	Data Transformation Services	数据传输系统
GPIO	General-purpose input/output	通用型之输入输出
IoT	Internet of Things	物联网
LPM	Low Power Mode	低功耗模式
MCU	Microcontroller Unit	微控制单元
SDK	Software Development Kit	软件开发工具包
SPI	Serial Peripheral Interface	串行外设接口
UART	Universal Asynchronous Receiver/Transmitter	通用异步收发传输器
USB	Universal Serial Bus	通用串行总线